Public Mathematical Models for Mean-Reversion, Momentum, and Market-Making with FPGA Implementation Guidance

Executive Summary

Public sources do not disclose the proprietary alpha formulas used inside top quantitative firms; even when firms such as Citadel Securities and Jane Street Capital describe their businesses (market making / liquidity provision), they do not publish the exact production signals, forecast models, or parameter calibrations that drive real-time quoting and execution. What is available—and highly useful for a dual-FPGA deterministic trading testbed—is a deep academic literature describing the same strategy families and microstructure models that underlie much of modern systematic trading: mean-reversion via residual/cointegration models and band/threshold rules; momentum/trend following via moving-average/return-sign signals and volatility scaling; and market-making/limit-order-book models that couple inventory risk, order-arrival intensities, and price impact. Below is a concise map from these public models to FPGA-friendly signal formulations—including a hardware-implementable definition of Order Flow Imbalance (OFI) and a practical plan to move a Zynq UltraScale+ PL dataplane from 100 MHz to 200 MHz using standard AMD/Xilinx clocking and timing-closure methodology.

Mean-Reversion Strategy Family

Mean-reversion strategies are a broad class of "contrarian" signals that trade on the hypothesis that a price (or a relative-value spread) oscillates around a statistical equilibrium, often modeled as a stationary residual process after removing common factors.

High-quality references (public, English)

Statistical arbitrage in the US equities market — Marco Avellaneda; Jeong-Hyun Lee (2010). Models stock residuals (via PCA factors or ETF regressions) as mean-reverting processes and derives systematic contrarian signals; also presents a cointegration-style residual framework for pairs-like trades. URL:

https://traders.berkeley.edu/papers/Statistical%20arbitrage%20in%20the%20US%20equities%20market.pdf

Pairs Trading: Performance of a Relative Value Arbitrage Rule — Evan G. Gatev; William N. Goetzmann; K. Geert Rouwenhorst (1999 working paper; published version 2006). Empirically tests a canonical "pairs trading" approach using historical distance matching and simple trading rules, documenting persistent excess returns under realistic cost assumptions.

URL: https://www.nber.org/papers/w7032

Optimal Mean Reversion Trading with Transaction Costs and Stop-Loss Exit — Tim Leung; Xin Li (2014/2015). Formulates optimal entry/exit times for a mean-reverting spread modeled as an Ornstein–Uhlenbeck (OU) process, explicitly incorporating transaction costs and stop-loss constraints.

URL: https://arxiv.org/ abs/1411.5062

Optimal Pairs Trading: A Stochastic Control Approach — Saray Mudchanatongsuk; Steven Primbs; Wai Kin Wong (2008). Models the log-spread of a pair as OU and uses stochastic control to derive dynamic portfolio/ position

policies under a mean-reverting assumption (explicitly noting the proprietary nature of many hedge-fund implementations).

URL: https://skoge.folk.ntnu.no/prost/proceedings/acc08/data/papers/0479.pdf

Mean Reversion in Stock Prices: Evidence and Implications — James M. Poterba; Lawrence H. Summers (1988). Provides foundational econometric evidence and methodology around mean reversion (negative long-horizon autocorrelation) in equity indices across countries; helpful as background justification even if your FPGA demo operates at microsecond horizons.

URL: https://economics.mit.edu/sites/ default/files/publications/1-s2.0-0304405X88900219-main.pdf

Core models and signal formulations

A useful way to organize mean-reversion math (and map it to hardware) is by "what mean-reverts."

Residual / spread mean reversion (pairs / relative value)

Plain-English goal: model the relationship between two assets so that the leftover "mispricing residual" is mean-reverting.

Readable equation (cointegration-style residual): $\log(P_t / P_{t0}) =$

$$\alpha*(t - t0) + \beta*\log(Q_t / Q_{t0}) + X_t$$

Where: - P_t = price (or index level) of asset P at time t - Q_t = price (or index level) of asset Q at time t - t0 = reference start time - α = drift term - β = "hedge ratio" (how much Q explains P) - X_t = residual (the part not explained by drift + β*Q); assumed stationary / mean-reverting**

Trading rule (readable): - If X_t is "too high" above its typical mean treat P as rich vs Q **short residual** (e.g., short P, long Q in β proportion). - If X_t is "too low" below its typical mean treat P as cheap vs Q **long residual** (e.g., long P, short Q).

OU / AR(1) mean reversion (single state variable)

Plain-English goal: treat the spread (or residual) as a random process that is pulled back toward a long-run mean.

Continuous-time OU model (readable):

$$dX = \kappa*(\mu - X)*dt + \sigma*dW$$

Where: - X = spread/residual state - μ = long-run mean level - κ = mean-reversion speed (bigger = snaps back faster) - σ = noise scale - dW = Brownian noise increment - dt = small time increment

Discrete-time AR(1) equivalent (readable):

$$X_t - \mu = \varphi*(X_{t-1} - \mu) + \varepsilon_t \text{ , with } |\varphi| < 1$$

Where: - φ = persistence (closer to 1 means slower reversion) - ε_t = random shock

Band / z-score variants used in practice

Plain-English goal: trade only when the residual is far enough from "normal," measured in standarddeviation units.

Readable z-score: $z_t = (X_t -$

$\mu_{hat}) / \sigma_{hat}$

Trade condition:

trade if $abs(z_t) > \theta$

Where: - μ_{hat} = estimated mean of X - σ_{hat} = estimated standard deviation of X - θ = entry threshold (e.g., 2.0) - abs(·) = absolute value

Factor-residual mean reversion (multi-asset stat-arb)

Plain-English goal: remove common market factors (via PCA or regression) and trade the remaining idiosyncratic residuals as mean-reverting.

In practice, this means: 1. Estimate common factors (slow path). 2. Compute residual return per asset = actual return factor-predicted return. 3. Treat that residual as X_t in the same mean-reversion logic above.

FPGA-friendly simplifications

For an FPGA demonstration, the right simplifications preserve the **structure** of the model but remove expensive estimation

Treat parameters as configuration, not continuously estimated. For example: set β , μ , σ , thresholds, and OU/EMA coefficients via PS registers, while the PL dataplane only does constant-time updates and comparisons (mirrors the "calibrate offline, execute online" split).

Use incremental updates: EMA-based residual proxies act as a low-cost discrete mean-reversion filter, avoiding per-tick regression/cointegration.

Avoid division and square roots: replace "z-score thresholding" with a multiply-compare that does not require a division: - instead of $abs(z_t) > \theta$, - implement: $abs(X_t - \mu) > \theta * \sigma$ using fixed-point multiply (DSP48) and preloaded constants.

Event-based updates fit FPGA pipelines: update the residual X_t , EMA state, and band logic only when a quote event arrives, eliminating a separate time-sampling loop and simplifying deterministic cycle accounting.

Relevant public industry/official context

Two Sigma Investments publicly positions itself as using a “scientific approach” and data-driven methods across investment strategies; this supports the claim that systematic trading models commonly separate research/calibration from execution, even if the exact production signals are proprietary.

URL: https:// www.twosigma.com/businesses/investment-management/

Momentum and Trend-Following Strategy Family

Momentum (trend-following) strategies take the opposite bet of mean reversion: recent price movement is assumed to persist (for a time), so signals follow the sign and/or strength of a trend estimate. The academic literature distinguishes cross-sectional momentum (winners vs losers) and time-series momentum (trend in each instrument), both of which have well-known signal forms that are implementable in low-latency systems.

High-quality references (public, English)

Time Series Momentum — Tobias J. Moskowitz; Yao Hua Ooi; Lasse Heje Pedersen (2012). Documents statistically significant time-series momentum across futures, typically using past returns as signals with volatility scaling.

URL: https://www.sciencedirect.com/science/article/pii/ S0304405X11002613

A Century of Evidence on Trend-Following Investing — Brian Hurst; Yao Hua Ooi; Lasse Heje Pedersen (2017). Extends trend-following evidence back to 1880 and frames trend as a persistent style with crisisperiod convexity properties; useful as an industry-adjacent primary source for the realness of trend logic.

URL: https://www.aqr.com/Insights/Research/Journal-Article/A-Century-of-Evidence-onTrend-Following-Investing

Returns to Buying Winners and Selling Losers: Implications for Stock Market Efficiency — Narasimhan Jegadeesh; Sheridan Titman (1993). Canonical cross-sectional momentum result: ranking winners/losers over intermediate horizons yields abnormal returns.

URL: https://onlinelibrary.wiley.com/doi/ abs/10.1111/j.1540-6261.1993.tb04702.x

Momentum has its moments — Pedro Barroso; Pedro Santa-Clara (2015). Shows momentum crash risk is time-varying and that scaling exposure by risk measures can reduce crashes while improving risk-adjusted returns. URL: https://www.sciencedirect.com/science/article/abs/pii/S0304405X14002566 Time series momentum: Is it there? (representative challenge paper) — Dazhi Huang (2019). Re-examines time-series momentum and argues the effect may be weaker under certain testing assumptions.

URL: https://down.aefweb.net/WorkingPapers/w717.pdf

Core models and signal formulations

Momentum has several standard “signal primitives” that map well to FPGA datapaths.

Return-sign / lookback momentum (time-series momentum)

Readable signal: signal_t = sign(sum_{i=1..L}

r_{t-i})

Equivalent phrasing: signal_t = sign(cumulative_return over the last L periods)

Where: - r_{t-i} = return at lag i - L = lookback length - sign(x) = +1 if x > 0, 1 if x < 0, 0 if x = 0

Volatility scaling (readable):

w_t = signal_t / sigma_hat_t

Where: - sigma_hat_t = estimated volatility (risk proxy) - w_t = exposure sizing factor (larger when volatility is low)

Moving-average crossover (trend filter)

Readable trend estimate: trend_t = EMA_short(p_t)

- EMA_long(p_t)

Readable decision rule: signal_t =

sign(trend_t)

Where: - p_t = price (or mid-price) at time t - EMA_short = fast EMA (more reactive) - EMA_long = slow EMA (more stable)

Risk-managed momentum variants

A common practical variant is: reduce momentum exposure when risk is high (for example, using realized variance or other volatility forecasts). The exact estimator varies, but the structural idea is “scale down in high-risk states.”

FPGA-friendly simplifications

For a sub-microsecond deterministic pipeline, the most credible simplifications are:

Replace long lookbacks with dual-EMA state: keep EMA_short and EMA_long as constant-time state variables updated per quote (much cheaper than summing long windows).

Use a volatility proxy that avoids squares/roots. A common FPGA-friendly form is an EMA of absolute returns:

$$v_hat_t = \beta * abs(\Delta p_t) + (1 - \beta) * v_hat_\{t-1\}$$

Where: - $\Delta p_t = p_t - p_\{t-1\}$ - β = smoothing factor (0..1 range in fixed-point) - abs(·) = absolute value

Then optionally scale order quantity using a reciprocal approximation of v_hat_t (or a simple "bucketed" size rule) instead of dividing.

Event-based computation: update trend and volatility only on quote arrival, avoiding a separate time-grid sampling loop and keeping determinism clean.

Relevant public industry/official context

AQR Capital Management hosts an official research summary for the century-scale trend-following paper (industry-facing, primary-hosted reference).

URL: https://www.aqr.com/Insights/Research/Journal-Article/A-Century-of-Evidence-on-Trend-Following-Investing

Two Sigma publishes broadly on applying modern ML to investing (not specifically momentum), useful for supporting a regime-aware selection narrative.

URL: https://www.twosigma.com/articles/ai-ininvestment-management-2026-outlook-part-i/

Market-Making and Order-Book Strategy Family

Market-making and order-book strategies formalize how a liquidity provider sets quotes under inventory risk and uncertain order flow, and how execution interacts with market impact and microstructure. This is directly relevant to the roles described by both Citadel Securities (as a market maker) and Jane Street (as a liquidity provider), even though their exact internal quoting algorithms are proprietary.

High-quality references (public, English)

High-frequency trading in a limit order book — Marco Avellaneda; Sasha Stoikov (2006 preprint; widely cited and later published variants). Derives optimal bid/ask quotes for a dealer using a utility framework with diffusion mid-price and Poisson order arrivals; introduces reservation price and optimal spread expressions under simplifying assumptions.

URL: https://people.orie.cornell.edu/sfs33/LimitOrderBook.pdf

Optimal dealer pricing under transactions and return uncertainty — Thomas Ho; Hans Stoll (1981). Classic dynamic dealer model: stochastic demand arrivals and return risk imply state-dependent optimal bid/ask quotes; foundational for inventory-control thinking. URL: https://www.sciencedirect.com/science/ article/abs/pii/0304405X81900209

Bid, ask and transaction prices in a specialist market with heterogeneously informed traders — Lawrence R. Glosten; Paul R. Milgrom (1985). Canonical adverse-selection model showing why bid-ask spreads persist: quotes incorporate information risk; motivates toxicity/imbalance features.

URL: https:// www.sciencedirect.com/science/article/pii/0304405X85900443

Optimal Execution of Portfolio Transactions — Robert Almgren; Neil Chriss (2000/2001). Classic optimal execution framework: trade off expected impact cost against execution risk under simplified impact assumptions.

URL: https://www.smallake.kr/wp-content/uploads/2016/03/optliq.pdf

The Price Impact of Order Book Events — Rama Cont; Arseniy Kukanov; Sasha Stoikov (2014 journal publication; 2010–2011 arXiv versions). Shows short-horizon mid-price changes are well explained by order flow imbalance (OFI) with approximately linear relationship and depth-dependent slope. URL: https:// arxiv.org/abs/1011.6402

Core models and signal formulations

Inventory-aware reservation price and spread (Avellaneda–Stoikov family)

Plain-English goal: shift your quote center away from mid-price when your inventory is too long/short; widen spreads when risk is higher.

Readable reservation price adjustment: r(s, q, t)

$= s - q * \gamma * \sigma^2 * (T - t)$

Where: - s = current mid-price - q = current inventory (positive = long, negative = short) - γ = risk aversion parameter - σ = price volatility parameter - T - t = remaining time horizon

Interpretation: - If you are long ($q > 0$), you shift your “fair” price down (encouraging sells). - If you are short ($q < 0$), you shift your “fair” price up (encouraging buys).

Dealer pricing under stochastic demand (Ho–Stoll family)

Plain-English goal: choose bid/ask quotes as an optimal control problem where trades arrive randomly and inventory creates risk.

You can think of this family as producing a rule of the form: - quote center depends on inventory state spread depends on risk and arrival parameters

Adverse selection (Glosten–Milgrom family)

Plain-English goal: quoted spreads exist because some counterparties may be informed; quotes must compensate for that information risk.

This motivates modern “microstructure features” (imbalance, toxicity measures) even when the implementation is simplified.

Optimal execution / impact (Almgren–Chriss family)

Plain-English goal: schedule trades to balance expected market impact against price risk during execution.

In simplified implementations, this often becomes: - use smaller sizes / slower rates when volatility is high respect impact and risk budgets

FPGA-friendly simplifications

For your capstone’s “Exchange-Lite Trader” closed loop, FPGA-friendly market-making and execution logic typically means:

Use top-of-book only (best bid/ask and sizes), unless the link payload supports multi-level depth.

Parameterize arrival/impact models: treat intensity/impact parameters as constants (or slow-updated registers) rather than estimating them at line rate. This keeps dataplane latency fixed.

Implement a piecewise-linear quoting policy derived from reservation-price logic:

- Quote center:
- quote_center = s - K_q * q
- Spread:
- spread = S_0 + K_r * risk_hat

Where: - K_q = inventory skew gain - S_0 = baseline spread - K_r = risk scaling gain - risk_hat = a cheap risk proxy (e.g., EMA(abs(Δp)) or regime flag)

Event-based microstructure features (OFI, imbalance, cancellations) are naturally FPGA-friendly because they are computed per event/quote and avoid expensive window scans.

Relevant public industry/official context

Citadel Securities explains the role of market makers (two-way quoting, earning spread, providing liquidity — especially during volatility). URL: https://www.citadelsecurities.com/what-we-do/what-is-amarket-maker/

Citadel Securities hosts a market-structure whitepaper on market making, technology, and regulation (highlevel, not formulas). URL: https://www.citadelsecurities.com/news-and-insights/changesmarket-making-impact-technology-regulation-whitepaper/

Jane Street states it is a global liquidity provider and trading firm using quantitative analysis and market mechanics.

URL: https://www.janestreet.com/what-we-do/overview/

Jane Street publishes periodic “Global Market Structure” updates.

URL: https://www.janestreet.com/ global-market-structure-2026/

Two Sigma publishes an official regime-modeling piece supporting the strategy-switching narrative. URL: https://www.twosigma.com/articles/a-machine-learning-approach-to-regime-modeling/

Order Flow Imbalance for Hardware

Order Flow Imbalance (OFI) measures net buy vs sell pressure captured by order book events (limit submissions, cancellations, and trades) at the best quotes (and, in multi-level variants, deeper levels). The key concept is that OFI increases when demand strengthens (bid queue grows / ask queue shrinks / prices improve) and decreases when supply strengthens; cancellations and market orders are treated similarly because both remove queue size.

Formal definitions and common estimators

Event-based top-of-book OFI (scalar, "Level 1")

Think of each order-book event as creating two signed "queue change" numbers: - ΔW = bid-side contribution (best-bid price/size change) - ΔV = ask-side contribution (best-ask price/size change)

Then: - Event contribution: e = ΔW - ΔV - OFI over a window: OFI = sum(e over events in that window)

Bid-side update rule (readable, per event n)

Let: - b_new = new best bid price after event n - b_old = previous best bid price (event n1) - r_new = new best bid size after event n - r_old = previous best bid size

Compute ΔW as:

- If bid price **improves** (b_new > b_old): ΔW = r_new
- If bid price is **unchanged** (b_new == b_old): ΔW = r_new - r_old
- If bid price **worsens** (b_new < b_old): ΔW = -r_old

Ask-side update rule (readable, per event n)

Let: - a_new = new best ask price - a_old = previous best ask price - q_new = new best ask size q_old = previous best ask size

Compute ΔV as:

- If ask price **worsens** (a_new > a_old): ΔV = -q_old
- If ask price is **unchanged** (a_new == a_old): ΔV = q_new - q_old

- If ask price **improves** ($a_{new} < a_{old}$): $\Delta V = q_{new}$

Event and window aggregation

Event contribution:

$$e = \Delta W - \Delta V$$

Window aggregation for a bucket $(t_{k-1}, t_k]$:

$$OFI(t_{k-1}, t_k) = \text{sum of } e \text{ for all events } \tau \text{ in } (t_{k-1}, t_k]$$

Multi-level OFI (vector, "MLOFI")

Multi-level OFI repeats the same logic per depth level $m = 1..M$:

- compute e^m using the same rules at level m
- aggregate it over the time/window:

$$MLOFI^m(t_{k-1}, t_k) = \text{sum of } e^m \text{ over events in the window}$$

Time-bucket relationship to price change (readable)

A common empirical relationship states that over a bucket, mid-price change tends to be approximately proportional to OFI:

$$\Delta P \approx \beta * OFI$$

Where: - ΔP = mid-price change over the bucket - β = fitted slope coefficient

Hardware-implementable numeric example (top-of-book) Assume one quote

event happens in $(t_{k-1}, t_k]$.

Before the event: - Best bid: $b_{old} = 100.00$, size $r_{old} = 30$ - Best ask: $a_{old} = 100.01$, size $q_{old} = 25$

Event: - A new buy limit order improves the best bid to $b_{new} = 100.01$ with $r_{new} = 10$ - Ask is unchanged: $a_{new} = a_{old}$, and $q_{new} = q_{old} = 25$

Compute: - Bid improved $\Delta W = r_{new} = 10$ - Ask unchanged $\Delta V = q_{new} - q_{old} = 25 - 25 = 0$ - Event contribution $e = \Delta W - \Delta V = 10 - 0 = +10$ - If this is the only event in the bucket OFI = +10

Interpretation: positive OFI indicates net buy-side pressure (demand strengthened at the best bid).

FPGA implementation notes for OFI

Event-based computation is ideal: it is a small set of compares (price up/down/same) plus adds/subtracts on queue sizes. This maps cleanly into a single registered pipeline stage at 200 MHz if inputs/outputs are registered.

If you can only afford a "V0 OFI," a common simplification is the static imbalance proxy:

imbalance_simple = r_best_bid - q_best_ask

But note: this proxy does not capture cancellations and price-level shifts that event-based OFI captures.

Moving the PL Core from 100 MHz to 200 MHz

Moving from 100 MHz to 200 MHz is typically feasible in UltraScale+ PL for moderate-complexity arithmetic pipelines, but it requires an explicit clocking plan, well-defined CDC boundaries, aggressive pipelining/ registration, and tight timing-closure discipline. AMD's UltraFast methodology emphasizes that timing closure is easier when HDL structure and constraints are designed for iteration and convergence.

Clocking approach

The Zynq UltraScale+ Processing System IP provides multiple PL clocks ("FCLKs") to the fabric and configures them through the PS configuration; default PL clocking is around 100 MHz.

A robust approach (especially under PYNQ overlays) is: - Keep AXI-Lite / control plane at 100 MHz (stable PS + interconnect closure). - Generate a separate 200 MHz core_clk in the PL using an MMCM/Clocking Wizard. - Run the dataplane (feature engine selector strategy risk order build) on core_clk, and cross to/from AXI using CDC-safe structures.

The PYNQ community specifically notes that a Clocking Wizard is often the easiest way to get a reliable 200 MHz fabric clock in overlay workflows.

CDC boundaries

When you introduce a 200 MHz core and keep AXI/control at 100 MHz, treat the interface as a real CDC boundary: - Control-plane to dataplane: CDC-safe register staging or a small async FIFO for multi-bit configuration words if they can toggle while running. - Telemetry/status back to PS: snapshot counters into AXI using either (a) double-buffered registers with handshake or (b) async FIFO for bursty telemetry. Single-bit CDC (start/stop pulses): 2-FF synchronizer is acceptable; AMD recommends setting ASYNC_REG on synchronizer flops.

Pipeline staging guidance

Concrete trader-specific staging guidance: - Turn feature_engine into a conveyor belt: separate (i) mid/ spread math, (ii) EMA MACs, (iii) volatility and OFI updates, and (iv) selector comparisons into distinct registered stages. - Prefer DSP48E2 pipelines for MACs: use DSP inference and/or explicit register stages to avoid long LUT carry chains. - Minimize high-fanout control: register mode bits and enables per stage rather than driving large logic regions in one cycle.

Phased plan to reach 200 MHz safely

Phase A: 100 MHz correctness lock
Freeze functional behavior; add pipeline-valid telemetry to prove determinism.

Phase B: Dual-clock refactor (still 100 MHz core)
Introduce CDC boundaries (AXI/control core) but keep the core at 100 MHz until CDC is clean.

Phase C: Step to 150 MHz
Raise only the core clock; fix top failing paths iteratively via timing reports.

Phase D: Step to 200 MHz
Add one register stage at a time in the highest-slack offenders (e.g., MAC + compare in one cycle, wide muxes, long carry chains). Use DSP pipelining where possible and re-run implementation after each change.

Spec Integration Assets

Paste-ready paragraph in the style of your design specification

Board B implements a deterministic adaptive trading engine that combines three real families of quantitative behavior—mean-reversion, momentum (trend-following), and microstructure-aware market making—into a fixed-latency FPGA pipeline. Each incoming QUOTE updates a compact top-of-book state and drives a feature engine that computes mid-price, spread, short and long EMAs, a low-latency volatility proxy, and Order Flow Imbalance (OFI), where OFI is an event-based measure of net buy-versus-sell pressure derived from best-quote queue changes. Using these features, an algorithm selector chooses the active strategy with zero jitter: in calm markets the trader applies mean-reversion by fading deviations of mid-price from the short EMA; in directional markets it switches to momentum by trading in the direction of the short–long EMA trend; and in liquidity-stressed or adversarial conditions it adopts market-maker-style defensive behavior by widening or suppressing risk-taking orders when spread, volatility, or OFI indicate deteriorating microstructure. All candidate orders then pass through a single-cycle risk stage enforcing position, rate, and loss limits before the order manager stamps a deterministic timestamp and transmits an ORDER frame, enabling fully hardware-based round-trip latency measurement and regime-dependent behavior that is explainable, measurable, and repeatable.

Strategy comparison table

Strategy family	Latency sensitivity	Feature complexity	FPGA resource cost	Typical risk controls
Mean-reversion	Medium–High (edge decays quickly)	Low–Medium (EMA/deviation, bands)	Low	Position limits; loss halt; cooldown after fills
Momentum / trend	High (fast trend capture)	Medium (dual EMA + volatility scaling)	Low–Medium	Volatility throttle; max order rate; stop-loss / kill switch

Market-making / order-book	Very High (queue dynamics)	Medium–High (inventory, OFI, spread logic)	Medium	Inventory caps; adverse-selection filters; cancel/replace throttles

Mermaid flowchart for the dataplane

```
flowchart LR
  Q[QUOTE frame in] --> QB[Quote book update]
  QB --> FE[Feature engine: mid, spread, EMA_s, EMA_l, vol, OFI]
  FE --> SEL[Algorithm selector]
  SEL --> SB[Strategy bank: MR | MOMO | DEF/MM]
  SB --> RM[Risk manager: pos | rate | loss | enable]
  RM --> OM[Order manager: id + timestamp + frame build]
  OM --> TX[Link TX: ORDER out]
  RXF[FILL/REJECT in] --> PT[Position + cash + PnL]
  RXF --> LH[Latency histogram + stats]
  PT --> AXI[AXI-Lite status regs]
  LH --> AXI
```